

Pestaña 1

TALLER QA-T01

FUNDAMENTOS DEL ASEGURAMIENTO DE LA CALIDAD

Competencia: 38369 - Controlar la calidad del servicio de software de acuerdo con los estándares técnicos.

Resultado de aprendizaje: 593146

Duración: 4 horas

Modalidad: Trabajo colaborativo (equipos del proyecto)

Objetivo

Analizar los conceptos fundamentales del aseguramiento de la calidad del software y relacionarlos con el proyecto formativo, identificando oportunidades de mejora desde la perspectiva de un equipo de QA.

Contexto

Su equipo ha sido contratado como el nuevo **Departamento de Quality Assurance** de la empresa desarrolladora del proyecto que han construido desde los primeros trimestres.

Antes de iniciar el proceso de pruebas, el Director de Calidad solicita un diagnóstico conceptual para verificar que todo el equipo comprenda los principios fundamentales del aseguramiento de la calidad y pueda aplicarlos al proyecto.

Actividad 1. Analizando nuestro proyecto

Objetivo: Relacionar los conceptos de calidad con el proyecto desarrollado.

Completen la siguiente tabla utilizando ejemplos reales de su proyecto.

Concepto	Explique con sus palabras	Ejemplo de su proyecto
Calidad	Cumplir con los requerimientos establecidos por la empresa (lógica de negocio). Así como con los requisitos funcionales y no funcionales del sistema (documentación).	En nuestro proyecto formativo, hemos implementado el módulo de gestión de ordenes de servicio (permite al administrador gestionar las ordenes del taller y al técnico, únicamente las asignadas).
QA	Quality Assurance (Asegurar la calidad). Conjunto de procesos que se realizan antes de codificar y durante la	Antes de programar el módulo de Gestión de Órdenes, el equipo definió que todo formulario debía validar los campos obligatorios (cliente, placa,

	codificación con el fin de prevenir errores.	técnico asignado, tipo de servicio y descripción del problema) en el frontend antes de enviarlos al backend, para prevenir que se generaran ordenes con datos incompletos o inválidos.
QC	Quality Control (Control de la calidad). Conjunto de técnicas para verificar que el software cumple con los requisitos establecidos.	Realizamos pruebas de testeo para verificar que la contraseña quedará encriptada en la base de datos (comprobando que no quedará almacenada en texto plano, sino como un hash generado mediante bcrypt).
Testing	Técnica de QC realizada mediante pruebas en el sistema para verificar su correcto funcionamiento (Comprobar que lo documentado, por ejemplo, el botón de login o función hacen lo que exactamente está escrito).	Hemos realizado pruebas unitarias en el módulo de ordenes de servicio y detectamos que cuando se requiere modificar una orden que está en proceso, genera un error relacionado con la base de datos. La lógica de negocio es incorrecta, se debe desactivar el botón Editar e informar al usuario a través del mensaje "No se pueden editar ordenes que están En proceso".
Verificación	Es el propósito del QC, busca comprobar que el sistema cumpla con los requisitos y especificaciones definidos en la documentación.	Comparamos el requisito no funcional RNF-005 (Seguridad contraseñas) con el comportamiento del sistema (observamos que las contraseñas de todos los usuarios si son encriptadas).
Validación	Proceso que consiste en comprobar si el producto cumple con las necesidades del usuario final.	Nos dimos cuenta que el proyecto no satisface la necesidad del usuario final porque el rol cliente, no puede hacer seguimiento de sus ordenes de servicio.

Regla: No se aceptan definiciones copiadas de Internet o de una IA. La explicación debe estar relacionada con el proyecto desarrollado por el equipo.

Actividad 2. ¿Dónde estamos hoy?

Analicen el estado actual de su proyecto y respondan.

Calidad

1. ¿Qué características hacen que su software sea de calidad?

RTA:

- **Seguridad:** El sistema permite la encriptación de contraseñas mediante el hash de bycript.
- **Usabilidad:** Se implementó el uso de mensajes claros para el usuario (Ej. "No se pueden editar ordenes de servicio que están en proceso").
- **Gestión de roles o control de acceso:** Cada usuario tiene acceso a su módulo y ven o gestionan lo que les corresponde.

2. ¿Qué aspectos consideran que aún deben mejorar?

RTA:

- La lógica de negocio y experiencia del usuario al usar el sistema (falta implementar el módulo de notificaciones y a su vez el módulo de seguimiento de órdenes para el caso del cliente. Al registrarse la orden de servicio, al cliente se le debe notificar mediante el sistema (Información detallada) y al correo electrónico el estado de su orden y técnico asignado en cada fase del proceso.

QA

3. ¿Qué actividades de aseguramiento de la calidad realizaron durante el desarrollo del proyecto, aunque en ese momento no supieran que correspondían a QA?
- Prevenir que el rol administrador ingresara manualmente clientes en el formulario de creación de la orden. Es decir que primero se deben registrar los roles cliente (Modulo gestión de usuarios) luego las motos (módulo de gestión de motocicletas) y finalmente (módulo de gestión de ordenes de servicio), donde mediante un select se deben traer los clientes registrados y que tiene motocicleta registrada.
 - Prevenir que el usuario digitara manualmente el modelo de la motocicleta (El modelo se trae mediante un select a la tabla motocicleta).

QC

4. ¿Qué actividades realizaron para verificar que el software funcionara correctamente?
- Comparar cada requisito con el comportamiento del sistema. De esta manera pudimos determinar que hay que corregir lógica de negocio y funcionalidades actuales.

Testing

5. ¿Qué tipos de pruebas han ejecutado hasta el momento?
- Hemos ejecutado pruebas unitarias (probamos funciones y módulos específicos del sistema. Ej. Motocicletas).
6. ¿Cuáles consideran que aún hacen falta?
- Consideramos que hacen falta pruebas de integración y más pruebas unitarias.

Verificación

7. ¿Cómo comprobaron que el software fue construido correctamente?

RTA//

Comprobamos que el software fue construido correctamente mediante el análisis de la documentación y verificación de esta.

Validación

8. ¿Cómo comprobarán que el software realmente satisface las necesidades del cliente?

RTA// Comprobamos que el software cumple con las necesidades del cliente mediante el análisis de historias de usuario.

Actividad 3. Caso de análisis

Lean el siguiente caso.

El sistema presenta los siguientes problemas:

- Algunos usuarios no pueden iniciar sesión.
- Las consultas tardan más de un minuto.
- La aplicación solo funciona correctamente en Google Chrome.
- Los administradores pueden acceder a funciones restringidas para clientes.
- El cliente manifiesta que el sistema no cumple algunas necesidades del negocio.

Analicen el caso anterior y completen la siguiente tabla.

Situación	¿Corresponde a QA, QC o Testing?	Justifique su respuesta
-----------	----------------------------------	-------------------------

Login con errores	Testing	Los errores fueron detectados después de la fase de desarrollo, mediante pruebas funcionales. Esta es una técnica del QC.
Lentitud del sistema	Testing	Es un problema de rendimiento detectado mediante la entrada masiva de un grupo de usuarios.
Incompatibilidad con navegadores	Testing	Es un problema encontrado tras probar el funcionamiento en varios sistemas operativos. El tipo de prueba realizado es de compatibilidad (pertenece al tipo de prueba testing).
Problemas de permisos	Testing	Es un defecto en la autorización y seguridad del sistema. Se prueba ingresando al sistema con diferentes tipos de rol (pertenece al tipo de pruebas de testeo).
Requisitos del cliente no satisfechos	QA	Los errores técnicos fueron identificados al probar el sistema, pero la responsabilidad recae sobre el QA porque debió analizar los requisitos y entender la lógica del negocio.

Actividad 4. Verificación vs Validación

Para cada situación indique si corresponde a **Verificación**, **Validación** o **Ambas**.

Situación	Verificación	Validación	Justificación
Revisar el documento de requisitos antes de programar		x	En este proceso se revisa la lógica de negocio.
Ejecutar pruebas funcionales del Login	x		Se verifica si el login se conectó satisfactoriamente con DB (Comparando lo que

			dice la documentación con la funcionalidad del sistema).
El cliente prueba el sistema y confirma que satisface sus necesidades		x	El cliente comprueba si el software cumple con las necesidades del negocio.
Revisar el código fuente mediante Pull Request	x		Se verifica si el código tiene errores de lógica y sintaxis.
Comparar el sistema con los criterios de aceptación	x	x	Se comprueba si la documentación está acorde con el funcionamiento del sistema.
Revisar el diseño de la base de datos	x		Se verifica si el diseño establecido cumple con

Actividad 5. Costos de la no calidad

Imaginen que el proyecto ya fue entregado al cliente.

Ahora aparecen los siguientes problemas.

Problema	Consecuencia para el cliente	Consecuencia para la empresa	¿Cómo se habría evitado?
El Login deja de funcionar en producción	Insatisfacción del cliente, no puede acceder a ningún módulo.	Pérdida de credibilidad, y clientes. No habría control en las órdenes de servicio.	Mediante pruebas de regresión y pruebas de integración antes de cada despliegue a producción, junto con un ambiente de testing separado del de producción.
Se pierden datos de los usuarios	Pérdida de trazabilidad, ganancias de usuarios y desconfianza al cliente.	Pérdida de ganancias, clientes y posibles sanciones legales por incumplimiento en la protección de datos.	Implementando backups automáticos periódicos de la base de datos, restricciones de integridad

			referencial y validaciones en el backend para evitar pérdida o corrupción de datos.
El sistema es muy lento	Una mala experiencia de usuario con el sistema, abandono del sistema y pérdida de confianza en el servicio.	Pérdida de clientes por insatisfacción, problemas con el flujo del sistema y daño a la reputación del taller	Optimizando las consultas a la base de datos, implementando paginación en las tablas, usando índices en los campos más consultados y haciendo pruebas de rendimiento antes de producción
La aplicación presenta fallos de seguridad	Desconfianza del cliente al ingresar sus datos personales en el sistema, exposición de información sensible y abandono del servicio.	Pérdida de información importante de la empresa, pérdida de clientes, reputación del taller y posibles demandas.	Realizando pruebas de seguridad siguiendo las recomendaciones de OWASP, aplicando la norma IEEE 29119, encriptando datos sensibles como tokens y contraseñas, configurando CORS correctamente y usando ORM para evitar inyección SQL.
El cliente rechaza el producto	Insatisfacción con el servicio recibido, pérdida de tiempo y desconfianza hacia el taller para futuras reparaciones.	Pérdida de ingresos, daño a la reputación del taller, pérdida de clientes recurrentes y posibles reseñas negativas que afecten la captación de nuevos clientes	Realizando una pre-revisión detallada antes de crear la orden de servicio, manteniendo comunicación constante con el cliente sobre el estado de su motocicleta, documentando correctamente el problema y el trabajo realizado, y generando un ticket de servicio con evidencia fotográfica

			de los daños y las reparaciones realizadas
--	--	--	--

Finalmente respondan:

1. ¿Cuál de estos problemas sería el más costoso para su proyecto?
 - Sería si el sistema llegara a tener fallos de seguridad, ya que quedarían vulnerables datos importantes tanto de la empresa como de los usuarios registrados clientes, técnicos y administradores. Además del daño económico, podría generar demandas legales por incumplimiento en la protección de datos, lo cual aplicando la regla del 1-10-100 sería el costo más alto posible.
2. ¿Qué impacto tendría en la imagen de la empresa?
 - Generaría mucha desconfianza en los clientes, el taller perdería reputación y credibilidad, y sería muy difícil recuperar esa confianza una vez perdida. En el mundo digital una mala experiencia se comparte rápidamente y puede alejar clientes potenciales
3. ¿Qué actividades de QA podrían prevenir estos problemas?
 - Siguiendo las recomendaciones de OWASP para seguridad web, aplicando la norma IEEE 29119 para pruebas de software, encriptando datos sensibles, configurando CORS correctamente y usando ORM como Prisma para evitar inyección SQL. En nuestro proyecto implementamos varias de estas medidas durante el sprint de correcciones

Actividad 6. Aplicación al proyecto

Reúnanse como equipo y elaboren una lista de actividades de calidad que realizarán durante este trimestre.

Completen la siguiente tabla.

Actividad QA	Responsable	Evidencia
Elaborar Plan Maestro de Pruebas	Kevin Bastidas	
Diseñar Casos de Prueba	Kevin Bastidas	
Ejecutar pruebas	Kevin Bastidas	
Registrar defectos	Juan Camilo	
Elaborar informe de pruebas	Juan Camilo	
Proponer mejoras	Juan Camilo	

Pueden agregar otras actividades según su proyecto.

Actividad 7. Mapa conceptual

Con base en el análisis realizado durante el taller, elaboren un **mapa conceptual** donde relacione los siguientes conceptos:

- Calidad.
- QA.
- QC.
- Testing.
- Verificación.
- Validación.
- Mejora Continua.

El mapa debe incluir **ejemplos del proyecto**, no únicamente definiciones.

Link canva: [Mapa conceptual](#)

Herramientas sugeridas:

- Draw.io
- Canva
- Lucidchart
- Miro
- PowerPoint

Actividad 8. Cuadro comparativo

Elaboren un cuadro comparativo como el siguiente.

Concepto	Objetivo	¿Cuándo se aplica?	Responsable	Ejemplo del proyecto
Calidad	Garantizar que el software cumpla los requisitos del cliente y funcione de forma segura y confiable	Durante todo el ciclo de desarrollo — desde el diseño hasta la entrega:	Kevin Bastidas, Juan Camilo	Rocket Service cumple requisitos funcionales como gestión de órdenes, roles diferenciados y seguridad con token encriptado
QA	Prevenir defectos mejorando el proceso de desarrollo antes	Desde el inicio del proyecto, durante la planificación y el desarrollo	Kevin Bastidas, Juan Camilo	Usar Jira con SCRUM para planificar funcionalidades, Git Flow con

	de que los errores ocurran			ramas separadas y Pull Requests antes de hacer merge
QC	Detectar defectos en el producto ya construido mediante inspección y revisión	Después de desarrollar una funcionalidad, antes de entregarla	Kevin Bastidas, Juan Camilo	Verificar que los insumos listaran correctamente tras migrar a Prisma, y que no hubiera usuarios duplicados tras agregar UNIQUE en la base de datos
Testing	Ejecutar el software para encontrar errores antes de que lleguen al cliente	Durante y después del desarrollo, antes de cada entrega	Kevin Bastidas, Juan Camilo	Probar el flujo completo del técnico — ASIGNADA, EN REVISIÓN ESPERANDO REPUESTOS, EN PROCESO, FINALIZADA verificando cada transición y validación
Verificación	Confirmar que el sistema está siendo construido correctamente siguiendo los estándares definidos	Durante el desarrollo, en cada revisión de código o entregable	Kevin Bastidas, Juan Camilo	Pull Requests en GitHub donde el compañero revisaba el código antes de hacer merge, asegurando que siguiera los estándares del proyecto
Validación	Confirmar que el sistema construido cumple con las necesidades reales del cliente	Al finalizar una funcionalidad o al entregar el producto	Kevin Bastidas, Juan Camilo	La revisión del jurado validó que el sistema cumplía los requisitos — detectaron mejoras como encriptación del token, restricciones UNIQUE y configuración de CORS

Evidencias a entregar

Cada equipo deberá entregar:

- Taller completamente diligenciado.
- Mapa conceptual.
- Cuadro comparativo.
- Conclusiones del equipo.

Criterios de evaluación

Criterio	Porcentaje
Análisis y aplicación de los conceptos	30%
Relación con el proyecto formativo	25%
Calidad del mapa conceptual	15%
Calidad del cuadro comparativo	15%
Argumentación y conclusiones	15%

Conclusiones de equipo

CONCLUSIONES DE EQUIPO ROCKET SERVICE - Kevin Bastidas, Juan Camilo

Como equipo hemos analizado cada concepto fundamental para tener una excelente gestión de calidad del software para nuestro proyecto Rocket Service planeando las metodologías que tenemos que seguir para poder crear un software de calidad y así poder mandarlo a producción. Tenemos en cuenta conceptos como Calidad, teniendo que garantizar que el software cumpla los requisitos de la empresa, el manejo de QA y QC previniendo defectos mejorando el proceso de desarrollo antes de que ocurran y así mismo detectandolos con el producto ya construido mediante revisión e inspección. tener en cuenta el Testing durante el proceso para ir ejecutando el software y así poder corregir errores antes de que llegue a producción y por último pero no menos importante la verificación y la validación, ir confirmado que el sistema este siendo construido de la forma correcta siguiendo los estándares establecidos y al mismo tiempo que el sistema cumple con éxito las necesidades de la empresa.

Además de esto, debemos tener en cuenta que hay que añadir el módulo de seguimiento de órdenes para el caso del cliente. Así como, el módulo de notificaciones para todos los usuarios en el sistema.